Session 19 Teamwork — A Second Resource, and a Real Relation Query

Context

Work in your team (six teams, each team of four students). Live code installed Prisma, described `User` / `Workspace` / `WorkspaceMember` in `schema.prisma`, ran a real migration, and read a real user through Prisma Client via one new route. Nobody has shown you, live, how to fetch a parent record together with its own related records in one query.

Goal

A real `Project` model — the Session 19 relational teaching subset of `Project` needed for this first Prisma migration, built from `sql-practice/schema.sql`'s own real `projects` table — added to `schema.prisma`, migrated with zero data loss, plus a real new route that returns a real workspace together with its own owner, members, and projects in one call, using a real Prisma feature you research yourselves: `include`.

What to build

You already know how to add a model, a relation, a delete-behavior choice, and run `prisma migrate dev`. Research, then build:

- **`Project`**: references a workspace (`workspace_id`), a unique `key` per workspace, a `name`, a `description` — the real `sql-practice/schema.sql` `projects` table also has a real `issue_seq` column backing Block 4's own atomic issue-key transaction; that column is deliberately NOT part of this model yet (Session 20's own dedicated slice) — do not add it.
- **`ON DELETE`.** You already learned this real mechanism last session — reuse it, do not re-research it: `workspace_id` is an ownership-hierarchy reference (the same real category as `workspace_members.workspace_id`).
- **A real migration** — run it yourselves. Read the real generated SQL before moving on; it must contain no `DROP` / `ALTER COLUMN` anywhere, only new structure.
- **`include`** — research this in the real official Prisma documentation's own section on relation queries. Build a route that returns a real workspace together with its real `owner`, its real `members` (each with their own real `user`), and its real `projects`, all in one query — nobody has shown you how to nest relations like this.
- **Written synthesis required:** what, specifically, was missing from the result before you added `include`; and whether `Project.workspaceId` could have used `SetNull` instead of `Cascade`, and why or why not, in terms of the column itself.

Rules

Allowed: everything already shown live, plus everything you discover by reading the real official Prisma documentation on relation queries (`include` / `select`). `npx prisma studio` (optional) to browse the real database visually.

Not yet: no Prisma transaction/raw-SQL-interop syntax, no real issue-key counter generation — that mechanism is deliberately deferred to a later session, not yours to build early.

Forbidden: do not touch `User`, `Workspace`, or `WorkspaceMember` beyond adding the one new relation field `Workspace` needs for `Project` (`projects Project[]`). Do not touch any existing Mongoose module, controller, service, or route — `ProjectsModule` / `WorkspacesModule` / `IssuesModule` / `AuthModule` and everything inside them stay exactly as they already are. Your new route must live only inside `PrismaCheckController`.

Test it (design your own full proof — this is a starting point, not the complete list)

Seed one real workspace with at least one owner, one other member, and one project — through Prisma Client (`prisma.workspace.create({ data: { ..., members: { create: [...] }, projects: { create: [...] } } })`), not raw SQL.

inside api/

BASH

```
npx prisma migrate dev --name add_projects
npm run start
```

in a second terminal

BASH

```
curl http://localhost:3000/prisma-check/workspaces
curl http://localhost:3000/workspaces
```

Run both `curl` commands — the second one must return exactly what it already returned before today's teamwork even started.

Done when

- `Project` exists, migrated, matching the Session 19 teaching subset described above (not yet including `issue_seq`).
- `GET /prisma-check/workspaces` returns a real workspace with its real owner, members-with-user, and projects, all in one query, proven with real seeded data.
- `GET /workspaces` (the pre-existing Mongoose-backed route) is proven, by actually running it, to be completely unaffected.
- Your written synthesis covers both required points above.

Working time

About 35–45 minutes — researching and correctly nesting `include`, and actually proving the Mongoose route is untouched, both deserve genuine time.